Southern Report

VOL. 1, NO. 049

2026-03-30

FEATURES

Unshorten (expand) short URLs with Node.js

Luciano Mammino · Luciano Mammino (loige)

Short URLs have been an invaluable tool for social media marketing for so many years and we are now used to seeing them everywhere. Most of the credit probably goes to URL shorteners services like Bit.ly , Goo.gl , YOURLS and Rebrandly that popularised the concept and made easy for everyone to start creating short URLs.

When working with URLs in some automation scenarios like analytics, information crawling, data retrieval, etc. it can be important to resolve (or " unshorten " or " expand ") short URLs, which means retrieving the original long URL.

In this article, we are going to see how short URLs work and how we can "expand" them into their original URL.

How short URLs work

A short URLs is a regular URL that most of the time results very short by following a very simple format:

http:/\/ /

The shorter the domain and the id (often called also slash-tag), the shorter will, of course, be the URL (e.g. http://loige. link/b).

What happens behind the scene of an URL shortener service is that there is a big database table that contains a map of all the existing short URLs and the related full-length URLs.

A URL shortener service lookup table might look like the following:

Domain Id Url loige.link b https://www.nodejsdesignp atterns.com/ loige.link gh https://github.com/lmammino rebrand.ly luciano-tw https://twitter.com/loige ... ...

Note that in this example the table supports multiple domains and that the pair domain - id must be unique in the table.

URL shorteners have to expose a web server which will be reached through all the domains mapped (in fact, most of the URL shortener services — like Rebrandly — supports different domains or even the possibility to associate your own branded domain).

For every request that satisfies the short URL format, domain and id are extracted and a query is executed against the database to search for the corresponding long URL. Finally, if a match is found, the web server responds with a redirect message, sending the browser of the user to the corresponding long URL.

Resolving short URLs

URL shorteners need to talk with browsers so they have to work as any other web server adopting the HTTP standard. The following diagram illustrates how the HTTP protocol is used when a user clicks a short URL:

A user clicks on a short URL and the browser issues a GET request to that URL.

The request hits the short URL service server which executes a look-up on its database to find the associated long URL. The long URL is returned as a redirect response to the user browser which means an HTTP 301 Moved Permanently response using the header Location to specify the new (long) URL.

The browser automatically follows the redirect response issuing a new GET request to the long URL displaying the content of that URL to the user.

This is what happens when a user through a browser deals with a short URL, everything happens behind the scenes and the user won't even notice that there was an exchange with a short URL server.

Simple enough, right?

Resolving short URLs in Node.js

Now that we know how short URL services work, it shouldn't be hard to create a Node.js script that is capable of resolving any type of short URL.

```
var request = require ( 'request' )

var uri = 'http://bit.ly/2fR0xVj'

request (

{

uri : uri ,

followRedirect : false ,

},

function ( err , httpResponse ) {

if ( err ) {

return console . error ( err )

}

console . log ( httpResponse . headers . location || uri )

},

)
```

For simplicity, in this script, we are using the request module to perform the HTTP request. By default, this library follows redirects so we need to explicitly disable this

Rust infrastructure can be your infrastructure

Huon Wilson

Rust is a reasonably large project: the compiler and standard libraries are over 350kloc, built across nearly 40000 commits by the hands of around 900 contributors. Not only that: there are more than 30 other repositories in the rust-lang GitHub organisation that shouldn't fall by the wayside, and, for rust-lang/rust alone, there are often more than 100 pull requests landing and a dozen new contributors in a single week.

Update 2015-06-16 : Homu is now available online: homu.io

It is sometimes chaotic... often chaotic... with 1.0 quickly approaching , and there are definitely places where the core team (and the rest of the community) sometimes can't keep up, but getting code into rust-lang/rust is rarely one 0 , due to two critical robots:

an integration bot/pull request manager, Barosl Lee's Homu

a review assigner, Josh Matthews and Nick Cameron's Highfive

The former ensures the master tree is essentially always passing tests, on all first-class platforms: it won't let a patch land until it does. The latter makes sure that pull requests don't slip through the cracks, there's someone with power watching out for each one.

I've been wanting to experiment with using the bots for my own code, and yesterday I finally got around to it, and (with a small bit of assistance from the tool authors) got it all setup in a short time. I can say from experience that it isn't too hard to configure these tools to run on your own repos on GitHub, and they're not Rust-specific: they work with any repository. All you need is a public-facing server!

Update 2015-03-19 : Homu and Highfive have been deployed to the contain-rs organisation, as @FlashCat .

Also, Manish points out that there is actually an open Operations Engineer position for Mozilla Research, to manage the infrastructure of Rust and Servo.

Homu: "Not rocket science"

Homu is a reimplementation/extension of the original bors bot. Bors was implemented by Graydon Hoare (Rust's original designer) in 2013 (or so) to apply Ben Elliston's "not rocket science" rule to Rust:

The Not Rocket Science Rule Of Software Engineering:

automatically maintain a repository of code that always passes all the tests

The core idea is simple: the correct way for anyone (including core developers) to land code into rust-lang/rust is to submit a pull request to that repo. Someone on the reviewers whitelist will review the code and, once it looks good, write a "review approved" comment: @bors r+ 1 . Homu will take the pull request, merge it with master into a new branch, and submit that branch to a testing backend. If the tests pass, Homu fast-forwards to the merge commit and starts on the next patch in its queue .

The process of code review and landing gated on testing works wonders for Rust: only really subtly—or transiently—broken patches get into master, other forms of brokenness are eliminated before touching mainline at all and backing out/reverting of patches once landed is rarely needed. 2

Having tests always passing sounds great, right? It's even better because it's not hard to use Homu with your own repos: just follow the usage instructions (I could only get the git version to work). It supports two testing backends at the moment, Buildbot and Travis CI .

Barosl tells me that he is planning has released Homu-as-a-service , so it is likely to get even simpler in future adding it to your project is easy and we live in the future.

Even if you don't need the test handling, Homu is still useful: the queue pages are nice pull request summary panels, especially since they can be combined to display pull requests across multiple repos, e.g. the rust-lang Homu instance manages two repositories, cargo and rust , and there's a variety of ways to digest that:

.../queue/cargo

.../queue/cargo+rust

.../queue/all

> *Which brings me on to highfive: Highfive: "welcome! You should hear from @huonw soon.*
>
> Huon Wilson

I easily lose track of pull requests against my own repos, so I've registered a lot of my repos with my Homu instance, and the /all endpoint shows me everything I want to know.

Which brings me on to highfive:

Highfive: "welcome! You should hear from @huonw soon."

As I said, I easily lose track of pull requests against my own repos. My GitHub news feed and emails is very busy with all development and discussion in rust-lang/rust, so small pull requests in other repositories can get drowned out and lost. Fortunately, Rust (and Servo) itself had this problem and has made progress on it already: Nick Cameron built on Josh Matthew's script for greeting new contributors to create @rust-highfive , a bot that still says hi , but also manages assigning potential reviewers to PRs .

The bot randomly chooses a person (out of a small per-repo

Comparing k-NN in Rust

Huon Wilson

In my voyages around the internet, I came across a pair of blog posts which compare the implementation of a k -nearest neighbour (k -NN) classifier in F# and OCaml. I couldn't resist writing the code into Rust to see how it fared.

Rust is a memory-safe systems language under heavy development; this code compiles with the latest nightly (as of 2014-06-10 12:00 UTC), specifically rustc 0.11.0-pre-nightly (e55f64f 2014-06-09 01:11:58 −0700) .

The Rust code is a nearly-direct translation of the original F# code, the only change was changing distance to compute the squared distance, that is, a*a + b*b + ... (square root is strictly increasing, yo).

For clarity, all errors are ignored (that's the .unwrap() calls): the input is assumed to be valid and IO is assumed to succeed. I wrote a follow-up post describing how one would handle errors . Also, I made no effort to remove/reduce/streamline allocations.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 use std :: io ::{ File , BufferedReader };

struct LabelPixel { label : int , pixels : Vec }

fn slurp_file (file : & Path) -> Vec { BufferedReader :: new (File :: open (file) .unwrap ()) .lines () .skip (1) .map (| line | { let line = line .unwrap (); let mut iter = line .as_slice () .trim () .split (',') .map (| x | from_str (x) .unwrap ());

LabelPixel { label : iter .next () .unwrap (), pixels : iter .collect () } }) .collect () }

fn distance_sqr (x : & [int], y : & [int]) -> int { /\/ run through the two vectors, summing up the squares of the differences x .iter () .zip (y .iter ()) .fold (0 , | s , (& a , & b)| s + (a - b) * (a - b)) }

fn classify (training : & [LabelPixel], pixels : & [int]) -> int { training .iter () /\/ find element of `training` with the smallest distance_sqr to `pixel` .min_by (| p | distance_sqr (p .pixels .as_slice (), pixels)) .unwrap () .label }

fn main () { let training_set = slurp_file (& Path :: new ("trainingsample.csv")); let validation_sample = slurp_file (& Path :: new ("validationsample.csv"));

let num_correct = validation_sample .iter () .filter (| x | { classify (training_set .as_slice (), x .pixels .as_slice ()) == x .label }) .count ();

println! ("Percentage correct: {}%" , num_correct as f64 / validation_sample .len () as f64 * 100.0); }

(Prints Percentage correct: 94.4% , matching the OCaml.)

How's it compare?

I don't have an F# compiler, so I'll only compare against the fastest OCaml solution (from the follow-up post), after making the same modification to distance .

The Rust was compiled with rustc -O , and the OCaml with ocamlopt str.cmxa (as recommended), using version 4.01.0. I ran each 3 times (times in seconds) on these CSV files .

Lang 1 2 3

Rust 3.56 3.46 3.86

OCaml 13.9 14.7 14.1

So the Rust code is about 3.5–4× faster than the OCaml.

It's worth noting that the Rust code is entirely safe and built directly (and mostly minimally) using the abstractions provided by the standard library. The speed is mainly due to the magic of Rust's (lazy) iterators which provide very efficient sequential access to elements of vectors/slices, as well as a variety of efficient adaptors implementing various useful algorithms. These may look high-level and hard to optimise, but they are very transparent to the compiler, resulting in fast machine code.

Updated 2014-06-11 : the Rust code is not as fast as it could be, due to bugs like #11751 , caused by LLVM being unable to understand that & pointers are never null. benh wrote a short slice-zip iterator that may make its way into the standard library: he even used it to make the code 3 times faster.

In comparison, the OCaml code has had to manually write a few functions (for folding and for reading lines from a file), and contains two possibly-concerning pieces of code:

1 2 let v1 = unsafe_get a1 i in let v2 = unsafe_get a2 i in

It might be interesting to compare against this D code , but I can't get it to compile right.

What about parallelism?

I'm glad you asked! Rust is designed to be good for concurrency, using the type system to guarantee that code is threadsafe. As I said before, Rust is under heavy development, and currently lacks a data parallelism library (so there's no parallel-map to just call directly yet), but it's easy enough to use the built-in futures for this.

The code can be made parallel simply by replacing the main function with the following.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 /\/ how many chunks should the validation sample be divided into? (== /\/ how many futures to create.) static NUM_CHUNKS : uint = 32 ;

fn main () { use sync ::{ Arc , Future }; use std :: cmp ;

The Sized Trait

Huon Wilson

An important piece in my story about trait objects in Rust 0 is the Sized trait , so I'm slotting in this short post between my discussion of low-level details and the post on "object safety" .

Other posts in this series on trait objects

Peeking inside Trait Objects

Where Self Meets Sized: Revisting Object Safety

Sized is a (very) special compiler built-in trait that is automatically implemented or not based on the sizedness of a type. A type is considered sized if the precise size of a value of type is known and fixed at compile time once the real types of the type parameters are known (i.e. after completing monomorphisation). For example,

u8 is one byte,

Vec is either 12 or 24 bytes (platforms with 32 and 64 bit pointers respectively), independent of T ,

pointers like &T are sized too, on 64-bit platforms &T is either 8 or 16 bytes, for sized T and unsized T respectively. This may seems like the size isn't known, but the sizedness of T is always known at compile time, so the precise one of those options is also known.

Types for which the size is not known are called dynamically sized types (DSTs) , and there's two classes of examples in current Rust 1 : [T] and Trait . A slice 2 [T] is unsized because it represents an unknown-at-compile-time number of T s contiguous in memory. A Trait is unsized because it represents a value of any type that implements Trait and these have wildly different sizes; I discussed this in the previous post too . Unsized values must always appear behind a pointer at runtime, like &[T] or Box , and have the information required to compute their size and other relevant properties (the length for [T] , the vtable for Trait) stored next to that pointer.

Sized types are more flexible, since the compiler knows how to manipulate them directly: passing them directly into functions, moving them about in memory. Putting an unsized type behind a pointer effectively makes it sized. A Box trait object, like Box , is the closest one can get to handling a trait object as a normal value; the Box ensures sizedness (at the expense of an allocation) without fundamentally changing the ownership semantics of a normal value.

? Sized

The Sized trait gets some special syntax for use in bounds, at the moment: ? Sized . Such a bound is necessary because Sized is special: it is a default bound for type parameters in most positions, and so one needs some way to opt-in to a parameter not necessarily being sized.

1 2 3 fn foo () {} /\/ can only be used with sized T

fn bar () {} /\/ can be used with both sized and unsized T

This bound is particularly special because adding the ? Sized bound to a parameter T increases the number of types that can be used for T , whereas every other trait bound reduces it.

This unusual decision was chosen because of the increased flexibility of sized types, and some data (which I now can't find in the issue tracker) which indicated that most type parameters needed to be sized. That is, not having these defaults would result in many instances of T: Sized bounds in the standard library and elsewhere.

However, I believe this data did not consider using some form of inference (like lifetime elision) to try to guess when sizedness was likely to be needed, and some data Niko has been collecting apparently implies that such inference may make removing this special case significantly more palatable. (It may or may not be so palatable so as to be worth the breaking change...)

Comments:

/r/rust

Per the previous post , this post is designed to reflect the state of Rust at version: rustc 1.0.0-nightly (44a287e6e 2015-01-08 17:03:40 −0800) . ↩

There is the possibility that Rust will gain some form of "inheritance" , and Niko points out to me that Sized may play an important role there too: certain types (e.g. "base classes" in an conventional inheritance scheme) make sense to be unsized. ↩

The unsized string type str is usually considered a slice, since it is just a [u8] with the guarantee that the bytes are valid UTF-8. ↩

How to Setup a Scheduled Scala Spark Job

Curalate Engineering

Have you written a Scala Spark job that processes a massive amount of data on an intimidating amount of RAM and you want to run it daily/weekly/monthly on a schedule on AWS ? I had to do this recently, and couldn't find a good tutorial on the full process to get the spark job running. Included in this article and accompanying repository is everything you need to get your Scala Spark job running on AWS Data Pipeline and EMR .

This tutorial is not going to walk you through the process of actually writing your specific Scala Spark job to do whatever number crunching you need. There are already plenty of resources available (1 , 2 , 3) to get you started on that. The code template for setting up a Spark Scala job is available in this GitHub repo .

Assuming that you have already written your Spark Job and are only using the AWS Java SDK to connect to your AWS data stores, drop your code in the Main function of SparkJob.scala and run the deploy.sh script to upload the fat jar to your S3 bucket.

If you do take other dependencies, then it may take some extra work on your part. To run a Scala Spark job on AWS you need to compile a fat jar that contains the byte code for your job and all of the libraries it needs to run. This project already has the sbt-assembly plugin setup and a assembly-MergeStrategy set up to package the Spark, Hadoop, and AWS SDK together in the fat jar. If you need to add in other libraries that do not play well with each other, or are using a noncompatible version of Spark for this current repo, there are a few good resources available to help you through the needed build.sbt modifications.

Outside of the previously mentioned needed changes you need to set a few parameters in the deploy.sh script. Mainly the deploymentPath to your specific S3 bucket, adding a profile to the AWS CLI command to upload to your specific S3 bucket if it's private, and changing the resulting fat jar name if you please. The deploy script uses the AWS CLI to upload the fat jar to S3, so if you do not have it installed and configured you will need to go through the steps to do that before using the deploy.sh script.

Setting Up The Job On AWS

Once your fat jar is uploaded to S3 it's time to set up the scheduled job on AWS Data Pipeline.

Create AWS Data Pipeline

Log on to the AWS dash, navigate to the AWS Data Pipeline console, and click the Create new pipeline button.

Load The Spark Job Template Definition

Add a name for your pipeline and select the Import a definition source option. Included in the GitHub repo for the Spark template is a datapipeline.json file that you can import that contains a pre-defined data pipeline for your Spark job that should simplify the setup processes. The definition contains the node configuration to fire the pipeline off on a schedule and notify you via an AWS SNS alarm on a success or failure run. All of your needed configuration options have been parameterized to simplify this process.

Set the min parameter to the EMR step(s) option. Update the S3 path and fat jar name to point to the fat jar uploaded earlier with the deploy.sh script. Set EC2KeyPair so that the data pipeline has access to your EC2 instances. Select the master node instance type and the number of and type of core-nodes for the Spark cluster. Finally, we need to set the ARN for both the success and failure notifications for the Spark job so you will know if something goes wrong with your scheduled job.

Create AWS SNS

Open a new tab and navigate to the AWS SNS console. Click the Create Topic option.

In the pop-up window set a topic name and display name for your success alarm and hit the create topic button.

You will be navigated to the topic details page for your new SNS Topic. Click the create subscription button.

Here you can choose the type of notification you want, for this example, we will set up an email notification for this alarm. Place your email address in the Endpoint field and hit Create subscription .

Check your email inbox and confirm the subscription to your SNS alarm.

Once you have successfully confirmed your subscription in your email, you should see it listed in the Subscriptions table on the Topic details page for your SNS Alarm.

You will need to repeat this process for both of your success and failure alarms. On each of the Topic details pages for both of your alarms, you need to copy the Topic ARN value and paste it into their respective parameter fields in your Data Pipeline setup.

Schedule your Job

Once your ARN parameters are set for your alarms you can move on to scheduling when this pipeline will run. Set the cadence for when the job will run and make sure you set your starting and ending dates to something in the future. You can also set the job to run only on pipeline activation if you would rather manually start your job.

Once you have your parameters set and scheduled all you need to do is set an S3 bucket location for the logs from the pipeline executions and you can click Activate at the bottom of the page. This will run a check on all of your settings and confirm that everything should work. Some things you might run into here could be setting improper values for

Foobar, Blossoms, and Isomorphism

yifanlu · Yifan Lu

A friend recently invited me to participate in Foobar, Google's recruiting tool that lets you solve interesting (and sometimes not-so-interesting) programming problems. This particular problem, titled "Distract the Guards" was very fun to solve but I found no good write-ups about it online! Solutions exist but it is rather hard to understand how the author came upon the solution. I thought I might take a shot and go into detail into how I approached it–as well as give proofs of correctness as needed.

Disclaimer: If you are participating in Foobar (hello googler) or have aspirations to do so in the future, please stop here in the spirit of the challenge. It's well known that Google has a finite pool of problems so you will miss out if you just read the solution.

To begin, here is the problem statement:

Distract the Guards ==========

The time for the mass escape has come, and you need to distract the guards so that the bunny prisoners can make it out! Unfortunately for you, they're watching the bunnies closely. Fortunately, this means they haven't realized yet that the space station is about to explode due to the destruction of the LAMBCHOP doomsday device. Also fortunately, all that time you spent working as first a minion and then a henchman means that you know the guards are fond of bananas. And gambling. And thumb wrestling.

The guards, being bored, readily accept your suggestion to play the Banana Games.

You will set up simultaneous thumb wrestling matches. In each match, two guards will pair off to thumb wrestle. The guard with fewer bananas will bet all their bananas, and the other guard will match the bet. The winner will receive all of the bet bananas. You don't pair off guards with the same number of bananas (you will see why, shortly). You know enough guard psychology to know that the one who has more bananas always gets over-confident and loses. Once a match begins, the pair of guards will continue to thumb wrestle and exchange bananas, until both of them have the same number of bananas. Once that happens, both of them will lose interest and go back to guarding the prisoners, and you don't want THAT to happen!

For example, if the two guards that were paired started with 3 and 5 bananas, after the first round of thumb wrestling they will have 6 and 2 (the one with 3 bananas wins and gets 3 bananas from the loser). After the second round, they will have 4 and 4 (the one with 6 bananas loses 2 bananas). At that point they stop and get back to guarding.

How is all this useful to distract the guards? Notice that if the guards had started with 1 and 4 bananas, then they keep thumb wrestling! 1, 4 -> 2, 3 -> 4, 1 -> 3, 2 -> 1, 4 and so on.

Now your plan is clear. You must pair up the guards in such a way that the maximum number of guards go into an infinite thumb wrestling loop!

Write a function answer(banana_list) which, given a list of positive integers depicting the amount of bananas the each guard starts with, returns the fewest possible number of guards that will be left to watch the prisoners. Element i of the list will be the number of bananas that guard i (counting from 0) starts with.

The number of guards will be at least 1 and not more than 100, and the number of bananas each guard starts with will be a positive integer no more than 1073741823 (i.e. 2^30 −1). Some of them stockpile a LOT of bananas.

Languages =========

To provide a Python solution, edit solution.py To provide a Java solution, edit solution.java

Test cases ==========

Inputs: (int list) banana_list = [1, 1] Output: (int) 2

Inputs: (int list) banana_list = [1, 7, 3, 21, 13, 19] Output: (int) 0

Now I love a good story and I love a challenging problem but the two fit together like chocolate and eggplant parmesan but I digress. If you parse through the bananas and thumb wrestling, it is easy to see that this is a combinatorics problem. The first thing to do is to break the large problem into some smaller ones that can be pieced together. Here we see that a key piece is figuring out, for any two given guards, if they will go into an infinite loop or not. Once we figure that out, the second part is to find which guards can be paired into infinite loops such that a maximum number of guards end up in infinite loops. Let's solve the second part first.

Assume we have a predicate \(willLoop(x,y)\) for two guards, each with \(x\) bananas and \(y\) bananas that returns true if the pair will loop. Can we then pair up all the guards optimally so we have the most number of infinite loops? Note once we have this, the answer will be simple: just return the total number of guards minus the number of guards that are paired up into infinite loops.

What if we just brute-force and try to find every possible pairing? We take one guard and try to pair her with another guard and if they don't loop, we try pairing her with a different guard. This will find us a solution but how do we find a maximum one where the most number of guards are paired off? Well, we can then try to find every possible set of pairings. How long will that take? Let's say there are \(n\) guards. Then it will take \(O(n^2)\) time to find one set of pairings. To find every possible pairing, notice that once we pair off two guards, those two guards cannot be used to pair with anyone else. So for every pairing in every solution set of pairings, we can remove that particular pair, reassign the remaining pairings, and be left with another potential solu-

Email providers in Norway

Andersos · Finn.no Tech

It all started with a simple tweet:

Er det noen som har noe norsk statistikk på bruk av e-post (gmail vs. outlook vs. online vs. yahoo osv)? F.eks @labs_finn_no eller @NRKbeta ?

— Håvard Bergersen (@haavardb) March 7, 2014

“Does anyone have any Norwegian statistics on the use of email (Gmail vs. Outlook vs. Online vs. Yahoo etc.)? Eg @labs_finn_no or @nrkbeta .”

We have access to the data so if this will help someone why not post it. We have around 4 million email addresses in a database. These emails are not verified. The data was retrieved 2014-06-06 . To answer the question “Gmail vs. Outlook vs. Online vs. Yahoo etc.” we would say:

Microsoft 33,57%

Google 15,32%

Telenor 12,53%

Yahoo 4,31%

The lists under shows the domain, number registered and percent of total. At the end is the table of the more popular email service providers (this list is cut off at 2500). Watch this space in the future if this is information that interests you.

Popular services:

Microsoft 33,57% (sum of hotmail, live, msn and outlook)

hotmail.com (1106084 - 28.06%)

live.no (96052 - 2.44%)

hotmail.no (67602 - 1.71%)

msn.com (29675 - 0.75%)

live.com (15014 - 0.38%)

outlook.com (9142 - 0.23%)

Google 15,32% (sum of gmail and googlemail)

gmail.com (599293 - 15.20%)

gmail.no (2982 - 0.08%)

googlemail.com (1766 - 0.04%)

Telenor 12,53% (sum og online, frisurf, telenor and adsl)

online.no (444897 - 11.28%)

frisurf.no (41768 - 1.06%)

telenor.com (3903 - 0.10%)

adsl.no (3643 - 0.09%)

Yahoo 4,31%

yahoo.no (94029 - 2.39%)

yahoo.com (75772 - 1.92%)

Universities:

stud.ntnu.no (7688 - 0.20%)

ntnu.no (1421 - 0.04%)

stud.nhh.no (1008 - 0.03%)

student.bi.no (925 - 0.02%)

hiof.no (921 - 0.02%)

medisin.uio.no (820 - 0.02%)

student.sv.uio.no (746 - 0.02%)

ifi.uio.no (725 - 0.02%)

uit.no (666 - 0.02%)

stud.hib.no (557 - 0.01%)

studmed.uio.no (522 - 0.01%)

umb.no (411 - 0.01%)

uus.no (408 - 0.01%)

uis.no (407 - 0.01%)

Telcos :

online.no (444897 - 11.28%)

frisurf.no (41768 - 1.06%)

telenor.com (3903 - 0.10%)

adsl.no (3643 - 0.09%)

c2i.net (67080 - 1.70%)

broadpark.no (48232 - 1.22%)

lyse.net (25964 - 0.66%)

getmail.no (19497 - 0.49%)

chello.no (18485 - 0.47%)

tele2.no (12218 - 0.31%)

How the dotnet CLI tooling runs your code

Matt Warren (.NET)

Just over a week ago the official 1.0 release of . NET Core was announced, the release includes:

the . NET Core runtime, libraries and tools and the ASP. NET Core libraries.

However alongside a completely new, revamped, xplat version of the . NET runtime, the development experience has been changed, with the dotnet based tooling now available (Note : the tooling itself is currently still in preview and it's expected to be RTM later this year)

So you can now write:

dotnet new dotnet restore dotnet run

and at the end you'll get the following output:

Hello World!

It's the dotnet CLI (Command Line Interface) tooling that is the focus of this post and more specifically how it actually runs your code , although if you want a tl;dr version see this tweet from @citizenmatt :

Traditional way of running . NET executables

As a brief reminder, . NET executables can't be run directly (they're just IL , not machine code), therefore the Windows OS has always needed to do a few tricks to execute them, from CLR via C# :

After Windows has examined the EXE file's header to determine whether to create a 32-bit process, a 64-bit process, or a WoW64 process, Windows loads the x86, x64, or IA64 version of MSCorEE.dll into the process's address space. ... Then, the process' primary thread calls a method defined inside MSCorEE.dll. This method initializes the CLR, loads the EXE assembly, and then calls its entry point method (Main). At this point, the managed application is up and running.

New way of running . NET executables

dotnet run

So how do things work now that we have the new CoreCLR and the CLI tooling? Firstly to understand what is going on under-the-hood, we need to set a few environment variables (COREHOST_TRACE and DOTNET_CLI_CAPTURE_TIMING) so that we get a more verbose output:

Here, amongst all the pretty ASCII-art, we can see that dotnet run actually executes the following cmd:

dotnet exec –additionalprobingpath C:\Users\matt\.nuget\packages c:\dotnet\bin\Debug\netcoreapp1.0\myapp.dll

Note : this is what happens when running a Console Application. The CLI tooling supports other scenarios , such as self-hosted web sites, which work differently.

dotnet exec and corehost

Up-to this point everything was happening within managed code, however once dotnet exec is called we jump over to unmanaged code within the corehost application . In addition several other .dlls are loaded, the last of which is the CoreCLR runtime itself (click to go to the main source file for each module):

hostpolicy.dll

hostfxr.dll

coreclr.dll

The main task that the corehost application performs is to calculate and locate all the dlls needed to run the application, along with their dependencies. The full output is available , but in summary it processes:

99 Managed dlls ("Adding runtime asset..")

136 Native dlls ("Adding native asset..")

There are so many individual files because the CoreCLR operates on a "pay-for-play" model, from Motivation Behind . NET Core :

By factoring the CoreFX libraries and allowing individual applications to pull in only those parts of CoreFX they require (a so-called "pay-for-play" model), server-based applications built with ASP. NET 5 can minimize their dependencies.

Finally, once all the housekeeping is done control is handed off to corehost , but not before the following properties are set to control the execution of the CoreCLR itself:

TRUSTED_PLATFORM_ASSEMBLIES =

Paths to 235 .dlls (99 managed, 136 native), from C:\Program Files\dotnet\shared\Microsoft. NETCore. App\1.0.0-rc2-3002702

APP_PATHS =

c:\dotnet\bin\Debug\netcoreapp1.0

APP_NI_PATHS =

c:\dotnet\bin\Debug\netcoreapp1.0

NATIVE_DLL_SEARCH_DIRECTORIES =

C:\Program Files\dotnet\shared\Microsoft. NETCore. App\1.0.0-rc2-3002702

c:\dotnet\bin\Debug\netcoreapp1.0

0-rc2-3002702 APP_PATHS = c:\dotnet\bin\Debug\netcoreapp1.

Matt Warren (.NET)

Fuzzing the .NET JIT Compiler

Matt Warren (.NET)

I recently came across the excellent 'Fuzzlyn' project , created as part of the 'Language-Based Security' course at Aarhus University . As per the project description Fuzzlyn is a:

... fuzzer which utilizes Roslyn to generate random C# programs

And what is a 'fuzzer', from the Wikipedia page for ' fuzzing ' :

Fuzzing or fuzz testing is an automated software testing technique that involves providing invalid, unexpected, or random data as inputs to a computer program.

Or in other words, a fuzzer is a program that tries to create source code that finds bugs in a compiler .

Massive kudos to the developers behind Fuzzlyn, Jakob Botsch Nielsen (who helped answer my questions when writing this post), Chris Schmidt and Jonas Larsen , it's an impressive project!! (to be clear, I have no link with the project and can't take any of the credit for it)

Compilation in . NET

But before we dive into 'Fuzzlyn' and what it does, we're going to take a quick look at 'compilation' in the . NET Framework . When you write C#/VB. NET/F# code (delete as appropriate) and compile it, the compiler converts it into Intermediate Language (IL) code. The IL is then stored in a .exe or .dll, which the Common Language Runtime (CLR) reads and executes when your program is actually run. However it's the job of the Just-in-Time (JIT) Compiler to convert the IL code into machine code.

Why is this relevant? Because Fuzzlyn works by comparing the output of a Debug and a Release version of a program and if they are different, there's a bug! But it turns out that very few optimisations are actually done by the 'Roslyn' compiler , compared to what the JIT does, from Eric Lippert's excellent post What does the optimize switch do? (2009)

The /optimize flag does not change a huge amount of our emitting and generation logic . We try to always generate straightforward, verifiable code and then rely upon the jitter to do the heavy lifting of optimizations when it generates the real machine code. But we will do some simple optimizations with that flag set. For example, with the flag set:

He then goes on to list the 15 things that the C# Compiler will optimise, before finishing with this:

That's pretty much it. These are very straightforward optimizations; there's no inlining of IL, no loop unrolling, no interprocedural analysis whatsoever. We let the jitter team worry about optimizing the heck out of the code when it is actually spit into machine code; that's the place where you can get real wins .

So in . NET, very few of the techniques that an 'Optimising Compiler' uses are done at compile-time . They are almost all done at run-time by the JIT Compiler (leaving aside AOT scenarios for the time being).

For reference, most of the differences in IL are there to make the code easier to debug, for instance given this C# code:

public void M () { foreach (var item in new [] { 1 , 2 , 3 , 4 }) { Console . WriteLine (item); } }

The differences in IL are shown below ('Release' on the left, 'Debug' on the right). As you can see there are a few extra nop instructions to allow the debugger to 'step-through' more locations in the code, plus an extra local variable, which makes it easier/possible to see the value when debugging.

(click for larger image or you can view the 'Release' version and the 'Debug' version on the excellent SharpLab)

For more information on the differences in Release/Debug code-gen see the 'Release (optimized)' section in this doc on CodeGen Differences . Also, because Roslyn is open-source we can see how this is handled in the code:

All usages of the 'OptimizationLevel' enum

All usage of the 'ILEmitStyle' enum

In Debug builds, extra 'sequence points' are created (as shown above)

Extra field added to the the async/await 'State Machine' in Debug builds

In Release builds, some 'catch' blocks are discarded

In Debug builds, hoisted variables aren't re-used

Extra Attribute is inserted in Debug builds

This all means that the 'Fuzzlyn' project has actually been finding bugs in the . NET JIT, not in the Roslyn Compiler

(well, except this one Finally block belonging to unexecuted try runs anyway , which was fixed here)

How it works

At the simplest level, Fuzzlyn works by compiling and running a piece of randomly generated code in 'Debug' and 'Release' versions and comparing the output. If the 2 versions produce different results, then it's a bug, specifically a bug in the optimisations that the JIT compiler has attempted.

{ [(int) StatementKind . Assignment] = 0.57 , [(int) StatementKind . If] = 0.17 , [(int) StatementKind . Block] = 0.1 , [(int) StatementKind . Call] = 0.1 , [(int) StatementKind

Rust shenanigans: return type polymorphism

Luciano Mammino · Luciano Mammino (loige)

In this article, I will describe Rust return type polymorphism (a.k.a. generic returns), a feature that I recently discovered and that I have been pretty intrigued about.

I am seeing this feature for the first time in a programming language and at first glance, it did seem like some sort of built-in compiler magic, available only in the standard library. In reality, it is a generalised feature that you can use in your own code every day.

Keep in mind that I am still quite a beginner with Rust, so my description might not be the most accurate but I will try to make a point on why I like this feature and how it works by using some examples. Hopefully, you will find this topic as interesting as I did!

Ok, enough chit chat, let's get into it!

Return type polymorphism, what?!

Before trying to explain what return type polymorphism actually is, let me show you a couple of interesting examples that might look familiar to you if you have been playing with Rust already:

```
use std :: collections :: HashSet ;

fn main () {

let nums = [ 2 , 17 , 22 , 48 , 1997 , 2 , 22 ];

let nums_square_no_dups : HashSet = nums . iter (). map (| x | x * x ). collect ();

println! ( "{:?}" , nums_square_no_dups ); /\/ {2304, 484, 3988009, 4, 289}

}
```

Ok, did you see it? I mean, how in the world is that collect() figuring out that I want to "collect" values from an iterator into a HashSet of integers? Indeed, it is giving me exactly a HashSet of integers!

Not convinced yet? We can also change this example slightly and see what happens... Let's try with Vec :

```
fn main () {

let nums = [ 2 , 17 , 22 , 48 , 1997 , 2 , 22 ];

let nums_square : Vec = nums . iter (). map (| x | x * x ). collect ();

println! ( "{:?}" , nums_square ); /\/ [4, 289, 484, 2304, 3988009, 4, 484]

}
```

Ok that's not removing duplicates anymore and it's preserving the order of elements, but that's not the point! The point is that collect() is still giving us what we want. This time we are asking for a Vec and indeed we get a Vec rather than a HashMap .

Note that this code is almost the same as the previous one. We didn't change anything other than the type declaration (and a variable name, but that's irrelevant)!

Also, keep in mind that, while Rust can ofter infer your types and, most often, you don't have to provide explicit type definitions, when using collect() the type definition is necessary.

Let's see what happens if we try to remove Vec :

```
fn main () {

let nums = [ 2 , 17 , 22 , 48 , 1997 , 2 , 22 ];

let nums_square = nums . iter (). map (| x | x * x ). collect ();

}
```

This example won't to compile because collect doesn't know what is the return type, so we get this nice-looking error:

```
error[E0282]: type annotations needed

|

| let nums_square = nums.iter().map(|x| x * x).collect();

| ^^^^^^^^^^ consider giving `nums_square` a type
```

I hope you starting to get the point. Some functions like collect() can behave differently, based on the expected return type! The return type needs to be explicit, so the Rust compiler can pick the right behaviour for you.

Ok, this is one of the first things I learned in Rust when doing coding challenges and I have been thinking for a while "this is just some iterator magic and it probably works only for a few standard types" ...

Then, more recently I saw some other piece of code that was doing something like this:

```
fn main () {

let a : u8 = Default :: default ();

let b : i64 = Default :: default ();

let c : String = Default :: default ();

let d : ( u16 , usize ) = ( Default :: default (), Default :: default ());

dbg! ( a ); /\/ a = 0

dbg! ( b ); /\/ b = 0
```

psvsd: Custom Vita microSD card adapter

yifanlu · Yifan Lu

One thing I love about Vita hacking is the depth of it. After investing so much time reverse engineering the software and hardware, you think you would run out of things to hack. Each loose end leads to another month long project. This all started in the development of HENkaku Ensō . We wanted an easy way to print debug statements early in boot. UART was a good candidate because the device initialization is very simple and the protocol is standard. The Vita SoC (likely called Kermit internally as we'll see later on) has seven UART ports. However, it is unlikely they are all hooked up on a retail console. After digging through the kernel code, I found that bbmc.skprx , the 3G modem driver contain references to UART. After a trusty FCC search , it turns out that the Vita's 3G modem uses a mini-PCIe connector but with a custom pin layout and a custom form factor. The datasheet gives some useful description for each pin, and UART_KERMIT seemed like the most likely candidate (there's also UART_SYSCON which is connected to the SCEI chip on the bottom of the board, which serves as a system controller and a UART_EXT which is not hooked up on the Vita side). So finding a debug output port was a success, but with the datasheet in front of me, the USB port caught my attention. Wouldn't it be neat to put in a custom USB device?

On USB in the Vita

A quick aside on the various USB ports found in the different models of the Vita.

The top port on OLED models (commonly referred to as the "mystery port" and incorrectly referred to as a "hidden video out port") is a USB host. It is unknown if the port is enabled by default or how to enable it.

The bottom port on OLED models (sometimes called the "multiconnector") supports UDC (USB client) but can also enable USB host support. It is unknown how this switch is controlled, but I'm guessing the syscon is involved and it's likely USB OTG.

The microUSB port on LCD models have the ID pin connected which implies support for USB OTG or something like it. However, it is unknown how to activate this feature.

There is a USB type A port on PS TV.

There is also a USB to Ethernet chip in the PS TV for the Ethernet port that is connected to Kermit via USB.

The audio codec chip is connected to Kermit via USB for all models.

and of course, the 3G modem on OLED models is connected by USB. On Wifi only models, VDD to the unfilled mini-PCIe pad is missing a bridge. The USB D+/D- signals are also missing a ferrite bead under the adjacent shield. It is unknown if bridging these three locations will enable the USB port on wifi models or if extra work is needed.

Designing a microSD adapter

In order to become more familiar with hardware design as well as understand how USB works on the Vita, I thought it would be fun to create a custom Vita USB device that fits on the modem port. The main reason I chose this port aside from the other USB ports is that it is the easiest to build. It is just a matter of designing and fabricating a PCB, which is simple to do. In comparison, connecting to any of the other USB ports would require creating custom adapters, molding plastic, and dealing with mechanical issues. Creating an adapter for the external ports is also not exactly a usable solution as the Vita is supposed to be portable, and having to dangle a USB port is not something most people are willing to do. In addition, my custom Vita modem card can expose the UART port to work as a console output device (which started this whole project). For this first project, I wanted to build a microSD adapter. Vita memory cards are notoriously expensive, with 32GB cards retailing for $79.99 USD. In comparison, a microSD card with similar performance and capacity goes for $12 USD. Therefore, it would be immensely useful to use microSD cards as a USB storage replacement for the proprietary Vita memory cards.

Choosing parts

SD to USB ICs are pretty cheap and common–you find them in any USB SD adapter. A quick research shows that most cheap adapters use an Alcor or Genesys chip. There is also the MAX14500 series chip from Maxim that is no longer in production and the Microchip USB2244 chip. The documentation for the cheap Asia manufactured chips were lacking so I went with the USB2244 even though it is more expensive (I don't plan to mass produce it anyways). Microchip provides good documentation in comparison, complete with layout guidelines and a reference design. Unfortunately, I can't find an Eagle library for the USB2244 so I had to design it myself (using Sparkfun's tutorial).

Next, I needed an Eagle part for the Vita modem form factor. Luckily, I found a good part for mini-PCIe and was able to modify it to the custom size that Vita uses thanks to the drawing in the datasheet.

Next is connecting the parts together. Having no experience whatsoever, I turned again to Sparkfun's tutorials . Copying the reference design , I came up with a board with the microSD adapter and pin headers for the UART.

> *I created a second mini-PCIe based design–this time with a mini-PCIe socket on the card to act as a breakout board.*
>
> yifanlu

I learned board layout again from Sparkfun making sure to follow the design guidelines from Microchip . I also cheated by looking at the layout for the reference board and ensuring that relative distance between objects match from my design. The main challenge is in routing because of the

AI-generated tests as ceremony

Mark Seemann · Mark Seemann (ploeh blog)

On epistemological soundness of using LLMs to generate automated tests.

For decades, software development thought leaders have tried to convince the industry that test-driven development (TDD) should be the norm. I think so too . Even so, the majority of developers don't use TDD. If they write tests, they add them after having written production code.

With the rise of large language models (LLMs, so-called AI) many developers see new opportunities: Let LLMs write the tests.

Is this a good idea?

After having thought about this for some time, I've come to the interim conclusion that it seems to be missing the point. It's tests as ceremony, rather than tests as an application of the scientific method .

How do you know that LLM-generated code works? #

People who are enthusiastic about using LLMs for programming often emphasise the the amount of code they can produce. It's striking so quickly the industry forgets that lines of code isn't a measure of productivity . We already had trouble with the amount of code that existed back when humans wrote it. Why do we think that accelerating this process is going to be an improvement?

When people wax lyrical about all the code that LLMs generated, I usually ask: How do you know that it works? To which the most common answer seems to be: I looked at the code, and it's fine.

This is where the discussion becomes difficult, because it's hard to respond to this claim without risking offending people. For what it's worth, I've personally looked at much code and deemed it correct, only to later discover that it contained defects. How do people think that bugs make it past code review and into production?

It's as if some variant of Gell-Mann amnesia is at work. Whenever a bug makes it into production, you acknowledge that it 'slipped past' vigilant efforts of quality assurance, but as soon as you've fixed the problem, you go back to believing that code-reading can prevent defects.

To be clear, I'm a big proponent of code reviews. To the degree that any science is done in this field , research indicates that it's one of the better ways of catching bugs early. My own experience supports this to a degree, but an effective code review is a concentrated effort. It's not a cursory scan over dozens of code files, followed by LGTM.

The world isn't black or white. There are stories of LLMs producing near-ready forms-over-data applications. Granted, this type of code is often repetitive, but uncomplicated. It's conceivable that if the code looks reasonable and smoke tests indicate that the application works, it most likely does. Furthermore, not all software is born equal. In some systems, errors are catastrophic, whereas in others, they're merely inconveniences .

There's little doubt that LLM-generated software is part of our future. This, in itself, may or may not be fine. We still need, however, to figure out how that impacts development processes. What does it mean, for example, related to software testing?

Using LLMs to generate tests #

Since automated tests, such as unit tests, are written in a programming language, the practice of automated testing has always been burdened with the obvious question: If we write code to test code, how do we know that the test code works? Who watches the watchmen? Is it going to be turtles all the way down ?

The answer, as argued in Epistemology of software , is that seeing a test fail is an example of the scientific method. It corroborates the (often unstated, implied) hypothesis that a new test, of a feature not yet implemented, should fail, thereby demonstrating the need for adding code to the System Under Test (SUT). This doesn't prove that the test is correct, but increases our rational belief that it is.

When using LLMs to generate tests for existing code, you skip this step. How do you know, then, that the generated test code is correct? That all tests pass is hardly a useful criterion. Looking at the test code may catch obvious errors, but again: Those people who already view automated tests as a chore to be done with aren't likely to perform a thorough code reading. And even a proper review may fail to unearth problems, such as tautological assertions .

This doesn't prove that the test is correct, but increases our rational belief that it is.

Mark Seemann

Rather, using LLMs to generate tests may lull you into a false sense of security. After all, now you have tests.

What is missing from this process is an understanding of why tests work in the first place. Tests work best when you have seen them fail.

Toward epistemological soundness #

Is there a way to take advantage of LLMs when writing tests? This is clearly a field where we have yet to discover better practices. Until then, here are a few ideas.

When writing tests after production code, you can still apply empirical Characterization Testing . In this process, you deliberately temporarily sabotage the SUT to see a test fail, and then revert that change. When using LLM-generated

Log4j2 in production – making it fly

mick · Finn.no Tech

Now that log4j2 is the predominant logging framework in use at FINN.no why not share the good news with the world and try to provide a summary over the introduction of this exciting new technology into our platform.

let's just one logging framework

In beginning of September 2013 it became the responsibility of one of our engineering teams to introduce a "best practice for logging" for all of FINN.

The proposal put forth was that we can standardise on one backend logging framework while there would be no need to standardise on the abstraction layer directly used in our code.

The rationale to not standardising the logging abstractions was...

• nearly every codebase, through a tree of dependencies, already includes all the different logging abstraction libraries, so the hard work of configuring them against the chosen logging framework is still required, • the different APIs to the different logging abstractions are not difficult for programmers to go between from one project to another.

While the rationale to standardising the logging framework was...

• makes life easier for programmers with one documented "best practice" for all,

• makes it possible through an in-house library to configure all abstraction layers, creating less configuration for programmers, • makes life easier for operations knowing all jvms log the same way,

• makes life easier for operations knowing all log files follow the same format.

Log4j2 wins HANDS down

Log4j2 was chosen as the logging framework given... • it provided all the features that logback was becoming popular for, • between old log4j and logback it was the only framework that was written in java with modern concurrency (ie not hard synchronised methods/blocks), • it provided a significant performance improvement (1000-10000 times faster)

• it consisted of a more active community (logback has been announced as the replacement for the old log4j, but log4j2 saw a new momentum in its apache community).

This proposal was checked with a vote by finn programmers • 73% agreed, 27% were unsure, no-one disagreed.

when nightly compression jams

Earlier on in this process we hit a bug with the old log4j where nightly compression of already rotated logfiles were locking up all requests in any (or most) jvms for up to ten seconds. This fault came back to poor java concurrency code in the original log4j (which logback cloned). Exacerbated by us having scores of jvms for all our different microservices running on the same machines so that when nightly compression kicked in it did so all in parallel. Possible fixes here were to

> *Breakages here were: log events on separate lines, avoiding commas are the end of lines between log events, thread context in wrong format, etc.*
>
> mick

a) stop compression of log files,

b) make loggers async, or

c) migrate over quickly to log4j2.

After some investigation, (c) was ruled out because there was no logstash plugin for log4j2 ready and moving forward without the json logfiles and the logstash & kibana integration was not an option. (a) was chosen as a temporary solution.

ready, steady, go...

Later on, when we started the work on upgrading all our services from thrift-0.6.1 up to thrift-0.9.1, we took the opportunity to kill two birds with the one stone. Log4j2 was out of beta, and we had ironed out the issues around the logstash plugin.

We'd be lying if we told you it was all completely pain free, introducing Log4j2 came with some concerns and hurdles.

• Using a release candidate of log4j2 in production lead to some concerns. So far the only consequence was slow startup times (eg even small services paused for 8 seconds during startup). This was due to log4j2 having to scan all classes for possible log4j plugins. This problem was fixed in 2.0-rc2. On the bright side – our use of the release candidate meant we spotted early and getting the upcoming initial release of log4j2 to support shaded jarfiles, of which we are heavily dependent on.

• Operation's had expressed concerns over nightly compression, raised from the earlier problem around nightly compression, that even if code no longer blocked while the compressing was happening in a background thread the amount of parallel compressions spawned would lead to IO contention which in turn leads to CPU contention. Because of this very real concern extensive tests have been executed, so far they've shown no measurable (under 1ms) impact exists upon services within the FINN platform. Furthermore this problem can be easily circumvented by adding the Size-

Leaving the Tower of Babel

Morten Lied Johansen · Finn.no Tech

Here at FINN we use mostly Java for our day-to-day work, but we do have some applications and modules written in other languages. We currently have code in Java, Ruby, JavaScript, Objective-C and Scala, supporting things as diverse as testing frameworks and iOS-apps.

Recently there has been some discussion with regards to what we should be using for new projects, as each team of developers start suggesting that they should use something they think would be easier and more effective. The arguments against are based on how easy it would be to recruit new developers who know these new technologies, how well it would perform under our kinds of loads, and wheter the assumption that we can be more effective is actually true.

Our strategy is firm on this point, that experiments can be done, but for major applications serving the public, we should be using Java for the time being. We are however considering if it's time to take a second look at our chosen languages, and see if we should expand our portfolio. This strategy is founded on a wish to reduce the size of our technology portfolio, so that we don't have to spend time and effort to bring developers up to speed when they switch teams. There's also a higher chance that new hires will be familiar with Java, while other technologies would often require a period of training when they first start.

During these discussions, we created a quick poll and sent it out on our internal discussionboard. The poll asked questions like "How long have you worked as a developer?", "Which language do you use for your day-to-day work?", "If you were free to chose, which language would you use for your next project?" and "Which programming languages do you know?". Our definition of "know" was very open in this poll, lowering the bar to allow people who had maybe written a single example program, and understands a bit of code could check the box.

Out of the nearly 100 people that work with development, 56 answered. We can assume that the people who did answer, were the people who were interested in the topic, and might not be a representative selection, but the results are interesting none the less.

So.. what did we learn?

Being a poll made in the midst of a discussion, mostly for fun, this part of the poll wasn't framed in a way that gives us much useful information. We can say that the average developer who answered has worked for around seven or eight years, but with what looks like a reasonably fair spread across all groups. Almost as many with a couple years experience, as there are people with 10 to 15 years.

Primary language in day-to-day work

Being a predominatly Java shop, most of the respondents were using Java for their daily work. 35 out of 56 were Java-developers. 11 respondents worked with JavaScript daily, while four worked with Scala, three with Ruby, two with Objective-C, and finally one database-developer who worked mostly in T-SQL (The SQL-variant used in Sybase).

Most popular choice if allowed to chose freely

The most interesting part of the poll, with many interesting insights. This is also the most loaded part, as the results could easily short-circuit any discussion about future choice in language.

The bad news (or good, depending on your point of view), is that there was no clear answer from this section. Keep in mind that around 40 people didn't answer this poll, and could be assumed to be content with the current status-quo.

The biggest group was Java, not surprisingly. The surprising part is that it was only 16 people who would chose Java if they could chose freely. This is much lower than expected, but still make up the largest group. Of these, 13 people are already using Java today. 20 Java developers would like to use something else.

So, if Java was the largest group, at 16 people, what does that mean for the remainder of the results? Well, 10 people would have chosen Scala, while JavaScript, Ruby, Clojure and Groovy clock in at about four or five respondents wanting to use them. At the bottom of the pack we have Python and Objective-C with three and two respondents chosing them.

Six people were interested enough to answer the poll, but chose the non-commitant "I don't care, as long as I'm making good stuff for our users" option.

The number of possible answers here was a rather large selection of popular and not-so-popular-but-quite-well-known languages, so the fact that the list only includes eight languages is a sign that it's not a completely random selection. Still, quite a lot of discussion is needed before any one of those languages gets center stage.

A couple curious details found in this part of the poll includes the fact that the only people who would chose JavaScript are people who are already working in JavaScript every day. Groovy, Clojure, Objective-C and Scala have all managed to be chosen by a person who doesn't actually know the langauge. There's only three people who would switch to Java, if allowed to chose.

Which languages do we "know"?

As mentioned earlier, the bar for "knowing" a language was set quite low in this poll, to get more diversity in answers. This resulted in some interesting numbers.

Being primarily a Java-shop, it might not surprise anyone that a full 100% knows Java. A little over three fourths know

Setup nginx with HTTP/2 for local development (OS X)

Gregers Rygg · Finn.no Tech

HTTP/2 became an official standard in May earlier this year, and support is starting to land in servers already. The most recent, and very welcome addition, is nginx 1.9.5 [1] . What makes nginx extra awesome is that it's very easy to set up in front of any other HTTP 1.x server (or HTTP/2 for that matter). Server push won't work just yet, but at least we can start to test how multiplexing works. Multiplexing is said to eliminate the need to concatenate resources into bundles. At FINN.no we do multiple releases a day, and it just feels wrong that users have to download the whole 100+ kB JavaScript bundle after every release, even though most of the time only a few lines have changed. If we don't need to bundle anymore, the users only need to download the few scripts that had changed since their last visit!

HTTPS over TLS 1.2 is a requirement to use HTTP/2 . Service Workers also require secure connections , and probably other features soon .

This guide will help you set up nginx for local development on OS X, with proxy passing requests to your local server on port 8080 (or whichever port you prefer). In plain English, that means we put nginx in between your browser and your local development server. Your browser communicates securely over HTTP/2 to nginx, and nginx forwards the requests to your local server over unsecured HTTP/1.1.

Installing nginx

You have to compile nginx with the –with-http_v2_module configuration parameter, but Homebrew makes that a breeze. It's one of my favorite tools on OS X.

If you don't have Homebrew, see Install Homebrew further down.

To compile and install nginx with http2:

$ brew update # update list of packages $ brew install –with-http2 nginx

Test that the install works (make sure port 8080 is available):

$ sudo nginx $ open http://localhost:8080/

You should see a page with the title "Welcome to nginx!"

stop nginx $ sudo nginx -s stop

Now it's time to set up the https server with HTTP/2 enabled. Open /usr/local/etc/nginx/nginx.conf in an editor, and comment out the existing server section. Then copy-paste in the config below instead [2] . If your local server runs on a different port than 8080, you can change it in the proxy_pass URL.

server { listen 443 ssl http2 ; server_name localhost ; ssl on ; ssl_protocols TLSv1 TLSv1.1 TLSv1.2 ; ssl_certificate cert.pem ; ssl_certificate_key cert.key ;

location / { proxy_pass http://localhost:8080 ; proxy_set_header Host $host ; proxy_set_header X-Real-IP $remote_addr ; proxy_set_header X-HTTPS 'True' ; } }

To use https we need to generate a self signed certificate. It will give you a warning in the browser, but it works fine for local development. This command will generate the certificate [3] :

$ cd /usr/local/etc/nginx/ $ sudo openssl req -x509 -sha256 -newkey rsa:2048 -keyout cert.key -out cert.pem \ -days 1024 -nodes -subj '/CN=local.finn.no'

When asked for «Common Name», fill in the hostname you use locally. Most FINN.no developers use local.finn.no

To set up nginx to start automatically on boot, Homebrew Services can set it up very easily.

install Homebrew Services $ brew tap homebrew/services

It is required to run nginx as root to open ports under 1024, and we want it to run on port 443. Homebrew Services will set it up correctly just by using sudo.

Start nginx (and install to /Library/LaunchDaemons):

$ sudo brew services start nginx

Homebrew Services can also be used to stop and restart nginx

$ sudo brew services stop nginx $ sudo brew services restart nginx

Now you should be able to open https://localhost/ and nginx should proxy pass to your local development server.

If you get a certificate warning, then it is working. It is only because you're using the self signed certificate and not one signed by a certificate authority. Most browsers allow you to ignore the warning.

By adding the self signed certificate as a trusted certificate in System Keychain, we'll get the green lock icon [4] :

$ sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain /usr/local/etc/nginx/cert.pem

If it doesn't work, see the Troubleshooting section further down.

To see that you really are using HTTP/2 in Chrome, you have to open the Network tab in Developer Tools. Right click (or CTRL-click) the column heading above the network requests, then make sure Protocol is checked.

Then you should see HTTP/2 traffic show up as h2 in the protocol column.

A look at the internals of 'Tiered JIT Compilation' in .NET Core

Matt Warren (.NET)

The . NET runtime (CLR) has predominantly used a just-in-time (JIT) compiler to convert your executable into machine code (leaving aside ahead-of-time (AOT) scenarios for the time being), as the official Microsoft docs say :

At execution time, a just-in-time (JIT) compiler translates the MSIL into native code . During this compilation, code must pass a verification process that examines the MSIL and metadata to find out whether the code can be determined to be type safe.

But how does that process actually work?

The same docs give us a bit more info :

JIT compilation takes into account the possibility that some code might never be called during execution. Instead of using time and memory to convert all the MSIL in a PE file to native code, it converts the MSIL as needed during execution and stores the resulting native code in memory so that it is accessible for subsequent calls in the context of that process. The loader creates and attaches a stub to each method in a type when the type is loaded and initialized. When a method is called for the first time, the stub passes control to the JIT compiler , which converts the MSIL for that method into native code and modifies the stub to point directly to the generated native code . Therefore, subsequent calls to the JIT-compiled method go directly to the native code.

Simple really!! However if you want to know more, the rest of this post will explore this process in detail.

In addition, we will look at a new feature that is making its way into the Core CLR , called ' Tiered Compilation '. This is a big change for the CLR, up till now . NET methods have only been JIT compiled once, on their first usage. Tiered compilation is looking to change that, allowing methods to be re-compiled into a more optimised version much like the Java Hotspot compiler .

How it works

But before we look at future plans, how does the current CLR allow the JIT to transform a method from IL to native code ? Well, they say 'a pictures speaks a thousand words'

Before the method is JITed

After the method has been JITed

The main things to note are:

The CLR has put in a 'precode' and 'stub' to divert the initial method call to the PreStubWorker() method (which ultimately calls the JIT). These are hand-written assembly code fragments consisting of only a few instructions.

Once the method had been JITed into 'native code', a stable entry point it created. For the rest of the life-time of the method the CLR guarantees that this won't change, so the rest of the run-time can depend on it remaining stable.

The 'temporary entry point' doesn't go away, it's still available because there may be other methods that are expecting to call it. However the associated 'precode fixup' has been re-written or 'back patched' to point to the newly created 'native code' instead of PreStubWorker() .

The CLR doesn't change the address of the call instruction in the method that called the method being JITted, it only changes the address inside the 'precode'. But because all method calls in the CLR go via a precode, the 2nd time the newly JITed method is called, the call will end up at the 'native code'.

For reference, the 'stable entry point' is the same memory location as the IntPtr that is returned when you call the RuntimeMethodHandle. GetFunctionPointer() method .

If you want to see this process in action for yourself, you can either re-compile the CoreCLR source and add the relevant debug information as I did or just use WinDbg and follow the steps in this excellent blog post (for more on the same topic see 'Advanced Call Processing in the CLR' and Vance Morrison's excellent write-up 'Digging into interface calls in the . NET Framework: Stub-based dispatch').

Finally, the different parts of the Core CLR source code that are involved are listed below:

JIT Helpers for 'PrecodeFixupThunk'

PrecodeFixupThunk (i386 assembly)

ThePreStub (i386 assembly)

PreStubWorker(..)

MethodDesc:: DoPrestub(..)

MethodDesc:: DoBackpatch(..)

MethodDesc:: SetStableEntryPointInterlocked(..)

Note: this post isn't going to look at how the JIT itself works, if you are interested in that take a look as this excellent overview written by one of the main developers.

JIT and Execution Engine (EE) Interaction

The make all this work the JIT and the EE have to work together, to get an idea of what is involved, take a look at this comment describing the rules that determine which type of precode the JIT can use . All this info is stored in the EE as it's the only place that has the full knowledge of what a method does, so the JIT has to ask which mode to work in.

In addition, the JIT has to ask the EE what the address of a functions entry point is, this is done via the following

magit-insert-worktrees improves status buffers

Huon Wilson

The Magit package for Emacs is my Git UI of choice, and git worktrees are very convenient. They mesh up particularly well by adding the built-in magit-insert-worktrees to magit-status-sections-hook . With this, the magit status buffer shows a summary of the branch/HEAD/path of all worktrees, and allow jumping between them in a flash.

1 2 ;; Show all worktrees at the end of the status buffer (if more than one) (add-hook 'magit-status-sections-hook #' magit-insert-worktrees t)

A magit status buffer with all the usual features, plus the new 'Worktrees' section at the end: the outlined row with an absolute path is the worktree for the current status buffer, and the other two are elsewhere on disk. Hitting RET on any worktree switches to it.

This post is about a particular feature of the Magit package for interfacing to Git within Emacs . I can't say enough good things about Magit (it's both incredibly powerful but also accessibly surfaces & teaches so much of the raw git interface; even when I've been written code in a different editor, I'll still use Magit to interact with Git)... but I also am not going to excessively extol its virtues or explain all the details here, as others have already done so .

Git has a feature called worktrees , which are a nifty way to do work in parallel: different branches checked out in different directories, but all sharing a single .git directory. This means less disk space used on duplicate .git clones, but, more usefully, data like branches & stashes is shared.

There's lots of ways to use worktrees, and they're especially handy in the age of AI agents. At the most basic level, they're useful for working concurrently , like reducing context switching when balancing deep projects, code reviewing, and small fixes .

I know some people like to create a new worktree for each branch they work on, and then remove them once finished (or get their agents to do so). I personally generally have fixed long-lived worktrees 0 that I cycle branches through as required: for my main work repo, I'm up to name , name2 , ..., name5 at the moment... and if I ever need a 6th one, I'll just create it.

Finding the worktrees

In either case, I find it annoying to keep track of what worktrees are 'live'. There's workable solutions, but they're a bit of friction:

CLI: run git worktree list or some alias.

Magit: hit keybinding Z g in magit buffers to see a switcher.

The magit status buffer is the entry-point to a Git repo, and surfaces a lot of information that's handy to have immediately available, in collapsible sections, things like diffs of any uncommitted and branch summaries. It reduces friction by making git status / git diff /... automatic and interactive, and is customisable.

I wanted to apply that mindset to worktrees too, and so, recently, started writing my own worktree section inserter... but then thought to double check the Magit source code for anything similar, and found exactly what I wanted 1 !

Magit has a builtin magit-insert-worktrees function that inserts a section summarising all worktrees (or nothing, if there's only one worktree). It's interactive : hitting RET on any worktree shows the magit status buffer for it.

It's not used by default, so it needs to be explicitly enabled by adding to the hook that the status buffer executes to show all the sections. My init files now include:

1 2 ;; Show all worktrees at the end of the status buffer (if more than one) (add-hook 'magit-status-sections-hook #' magit-insert-worktrees t)

My status buffers now look like the screenshot above. Yay!

I've settled on long-lived worktrees for two reasons:

IME many code-bases have at least a little bit of static setup that's intentionally not tracked in Git (like initialising an .env file from an .env.example file), and starting a new worktree requires redoing this... this can be solved with scripts, or just be side-stepped.

I have an (arguably bad) habit of taking notes, leaving reference data and/or creating ad-hoc scripts in untracked files in a worktree, and sometimes want to return to them and persist them later (commit, or paste into a issue tracker, or similar), so creating then deleting worktrees risks accidentally losing them.

↩

This function appears to have existed since 2016, but not be mentioned in the manual, so I submitted an addition . ↩

Going all out with the Strap-on Project

espen · Finn.no Tech

In January we started on an ambitious project called “the Strap-on project”. The infantile name aside this project is about rewriting our entire front-end tier using Object Oriented CSS (OOCSS) , written by Nicole Sulivan , as the secret sauce to achieving the desired results.

Why OOCSS?

At FINN we have teams grouped by business units and all of them have developers who contribute with code on our site. This is provided us with a head ache with CSS, because there are no clear idioms for how to best write CSS. In our case this had cause teams to create their own “islands” of CSS by using # trails, train wreck selector classes and !important-wars. Teams had duplicate CSS for the same layout and we had a hard time keeping the user experience consistent across different parts of our service.

We had way too much CSS code in total and we sent loads of CSS on each page request with just a small percentage actually being used. This made page load slow and rendering slow. Page rendering speed is equal to money, so we had to change something.

OOCSS - making CSS coding a thing of the past

OOCSS is a framework on which to build your own. It provides with a base set of modules and concepts which is essentials to building stuff in HTML with CSS. Grids, modules, lines, etc. These base modules provides an abstraction on top of CSS which makes authoring of CSS a thing of the past. In order to layout basic pages all we do is to use the building blocks and put together what ever you want. Basic boring stuff such as clearing and the box-model is no longer anything you need to worry about, it’s taken care of. This enables developers to focus upon fulfilling business requirements instead of battling CSS differences in browser time and time again.

Why not LESS, Compass or stuff like that?

The CSS language abstractions that exist out there are all very cool projects and make a whole lot of sense to use for a lot of projects. However, in our case choosing one of these tools right now would not provide us with some of the benefits we are looking for.

At FINN we have a problem that we are duplicating CSS across our development teams. This makes creating a consistent user experience and making global changes harder than it should be. Using an abstraction would help us hide some these problems, but would not solve the underlying issue which is that we create too much code. OOCSS and its rigorous set of rules helps reduce the amount of code drastically and provides rigid rules which prevents duplication across teams. What we are looking for are:

Improve rendering speed

Improve development speed

Easier to provide a consistent user experience

A touch of Bootstrap too

The engineers over at Twitter has created an amazing framework, Bootstrap , which is hugely popular all over the world. We have indeed paid close attention to how they have done and we are very much inspired by their work. However, going all out and just adopting Bootstrap was not an option. We feel it is too bloated and it does not provide the speed and performance benefits OOCSS gives. Having said that, a lot of our setup with forms and form elements is influenced by how Bootstrap does things.

Half way, how does it look?

In short,iIt looks pretty darn impressive! We have removed more than twenty CSS files and reduced the amount of CSS code lines with more than thirty thousand (this does say quite a bit of the mess we where in, I know). We have migrated the motorized vehicles, jobs, real estate sections and the front page.

Number of CSS files

130

38

Lines of CSS code

32 789

2 927

Lines of section specific CSS

727

89

These are pretty impressive results and very much in line with what Nicole is talking about in her presentations about OOCSS and performance.

is it all singing, all dancing?

Of course not! The responsive bit is something we are looking at reworking. Team Oppdrag has created one way of solving this and Team Reise another. For the Strap-on project we will probably end up with something in between. There are some things you need to take into consideration when choosing an approach: How much control do you have over the markup being written? Or to put it in another way, how many people are working with the same code? The more people, the harder it is to apply very strict rules as people will be “tourists” in the code base and might have a hard time getting it.

Another big challenge is to provide enough support and

The modern way to draw squircles using corner-shape in CSS

Amit Merchant

For years, we've faked "characterful" corners with SVG masks, pseudo-elements, and more than a few headaches. The problem? Border-radius only controls size, not shape.

The experimental (and fairly new) corner-shape property changes all that by letting us define the actual shape of an element's corners directly in CSS. Squircles! I'm looking at you .

With a single declaration, we can turn convex curves into concave scoops , carve precise notches , or dial in superellipse geometry , while borders, shadows, and backgrounds follow the cut. It's progressive enhancement in the truest sense: a new layer of polish, no hacks required.

What is the corner-shape property?

How it differs from border-radius

Some examples

What about fallbacks?

Best practices

What is the corner-shape property?

The corner-shape is an experimental CSS shorthand that defines the actual shape of a box's corners within the area established by border-radius .

It can use keywords like round , scoop , bevel , notch , square , squircle , or a numeric superellipse() function. Backgrounds, borders, outlines, shadows, overflow, and backdrop-filter will follow the defined corner shape.

.box { width : 200px ; height : 200px ; border-radius : 32px ; corner-shape : squircle ; }

One thing to keep in mind is that the property only has an effect when a non-zero border-radius is present. Without it, the corners will remain square regardless of the corner-shape value.

And like many CSS properties, corner-shape is a shorthand that can accept up to four values to define different shapes for each corner individually.

.box { corner-top-left-shape : scoop ; corner-top-right-shape : notch ; corner-bottom-right-shape : squircle ; corner-bottom-left-shape : round ;

/* or using the shorthand */ corner-shape : scoop notch squircle round ; }

It supports smooth animation between shapes because keywords map to superellipse() equivalents, and browsers constrain opposite corners to avoid overlap.

How it differs from border-radius

The border-radius sets the corner's size (how far it rounds). While the corner-shape sets the corner's geometry (what kind of curve or cut it is).

For instance, border-radius: 30px with corner-shape: round behaves like classic rounded corners. Changing to scoop inverts the curve into a concave cut; bevel draws a straight chamfer; notch creates an inset; squircle makes superellipse-style corners—all while the radius still controls extent.

Apart from this, corner-shape: round matches the normal border-radius look; corner-shape: square effectively cancels the rounding even if a radius is set.

Some examples

Here's a graphic showing different corner-shape values applied to boxes with the same border-radius .

As you can tell, each shape gives a distinct visual style while respecting the same radius size. This opens up new design possibilities without complex SVGs or masks.

Try this CodePen in a latest Chromium-based browser to see it in action.

Also, try this website to play around with different corner-shape values interactively: squircle.style

What about fallbacks?

Since corner-shape is still experimental (currently only available in Chrome 139 and above or similar Chromium forks) and not widely supported yet, it's important to provide fallbacks for browsers that don't recognize it and make it a progressive enhancement.

Here are safe, production-ready patterns to use corner-shape with a border-radius fallback. The idea: write classic border-radius first, then layer corner-shape inside @supports so unsupported browsers keep the rounded corners.

Basic keyword shape fallback

.card { /* Fallback everyone supports */ border-radius : 24px ;

/* Only apply corner-shape where supported */ @supports (corner-shape : round) { /* round is the default; use a different shape */ corner-shape : scoop ; /* concave corners */ } }

Per-corner values with fallback

.badge { border-radius : 16px ; /* fallback */ }

@supports (corner-shape : bevel) { .badge { /* TL, TR, BR, BL shapes (clockwise) */ corner-shape : bevel notch round squircle ; } }

The great big Schibsted programming language survey 2015

Morten Lied Johansen · Finn.no Tech

During September, I collected answers on a survey about programming languages from people who work in one of the many companies that make up the Schibsted family. The survey is neither requested, sanctioned, approved or even suggested by anyone in management, in any Schibsted company. It is entirely my own personal pet-project, because I'm interested in these things.

The survey opens with the following introduction:

In 2012, there was a discussion about technology choices, and particulary programming languages to use in FINN. As always in such discussions, claims like "It's impossible to hire people who know language X, so we need to always use Java" or "We can't expect programmers to learn a new language just because we think language Y would be a better fit" were put forward.

Personally, I've never liked those arguments, because they contradict what I think defines a "good programmer", and I'm optimistically hoping that we only hire "good programmers". Also, instead of opinions, beliefs, rumours and hearsay, let's be data driven in these discussions! So I decided I wanted to put it to the test, and created a survey and posted it for all the company to see.

The results from that survey were blogged about here: http://tech.finn.no/2012/09/04/leaving-the-tower-of-babel/

Now, almost three years later, a lot of changes are happening. FINN is becoming more close with it's Schibsted brethren, Schibsted itself is turning towards a more technological outlook, and I felt it was time to get an updated view, and why not involve all of Schibsted while we're at it.

So, "The great big Schibsted programming language survey 2015" was created, and hopefully it will make the rounds in all of Schibsted and gather some interesting, fun, surprising and (with a bit of luck) useful insights.

About the responses

309 people responded to the survey, which is more than many of my co-workers thought would respond.

Over 50% of respondents are in their thirties, and if we look at the ages from 26 to 40, we cover over 75% of respondents. On gender, it's even worse, with a whopping 93.5% of respondents admitting to being male. We also seem to shun inexperienced people, with 42% having 10 years of experience or more, and only 2.3% of respondents fresh from school.

It seems we have some work to do with regards to diversity.

According to Arena (The Schibsted-wide intranet), there are just under 125 companies in the Schibsted familiy. In those companies, 9355 people are employed.

That means somewhere between 20% and 30% response rate in the target group, which isn't too bad.

Unfortunately, our results are not representative. FINN is by no means the largest company, but supplied the most answers with 80 respondents. The second largest group was the various incarnations of Schibsted Product & Technology (SPT), which supplied 53 responses if we combine them all. Those two supplied over a third of all responses. Third place belongs to Schibsted Tech Polska (16 responses), which beat Wilhaben (15), Le Bon Coin (13), Blocket (12) and Aftonbladet (10) to close out the top 7.

This probably means that the results are more a description of FINN and SPT than a description of Schibsted in general.

Those 309 people listed 103 different companies, which is about 20 less than the total number of companies. That sounds great! Except I'm fairly certain that's not what I have. Trying to normalize the data entered here, fixing variations in spelling and other issues I'll get back to in a minute, I reduced it to 45 companies, one of which was Schibsted itself.

Those issues are companies that don't exist (atleast according to Arena)! I've tried my best to guess which company was really intended, but might have made mistakes here and there. In some cases, I had no idea which company to use instead, so I just left it as is, even if the company is not listed as part of Schibsted. Some examples: Schibsted Tech Madrid, SPT - Madrid, SCM Spain, SCM Central, Schibsted Norge Tech, Plan3 and Schibsted ATE. None of these were listed anywhere, but I've tried my best to place them some logical place.

Another problem was SPT aka Schibsted Products & Technology. SPT is techically four companies, SPT Norway, SPT Sweden, SPT Spain and SPT UK. Most people in SPT simply answered SPT, but they estimated as little as 20 people working in SPT, and as high as 250. I've disregarded their disagreement, and combined them all to just SPT.

Two people said they work in Schibsted, but they had wildly differing opinions about how many people work in Schibsted. One said 10000 total, 1500 technical, the other said 1000 total, 200 technical.

Disregarding the problems here, let's look at the numbers entered. Using the averages for each company and summing up, we arrive at roughly 4500 people working in those 47 companies. For technical positions, the sum of averages is roughly 1600. Since we have responses from about half the companies, that fits nicely with the total being just under 10000.

Languages in regular use

The most commonly used language, if you can call it that, is Bash. 52.1% of respondents use Bash regularly. Close behind, at 50.2% is JavaScript. Neither of these are surprising, considering the business we're in. Java clocks in at 49.2%,

Backup your server with Dropbox

Luciano Mammino · Luciano Mammino (loige)

In my early days as CTO at Sbaam I had to setup a web server from the ground up. As it happens in many startups the work had to be done quickly and with an almost-0-budget , so it left no space to sophisticated solutions for recurring tasks such as backup . I always have been a web developer and focused on coding so, I admit I had really a poor knowledge about how to setup a remote unix virtual machine.

So, speaking about backups, I needed a solution that would be cost-effective, easy to install and easy to maintain at the same time. I would have loved it if it can be as simple as sharing a Dropbox folder. As this thought crossed my mind I wondered if there was some way to interact with dropbox from a script to create files and folders and started googling about it. Luckily Dropbox offers a good command line client that allows to bring your synced files and folders also on graphic-less machines.

Ultimately my solution was to install the Dropbox command line on the server using a dedicated Dropbox account and backup files by simply copying/linking them on the Dropbox folder. This way I prepared and scheduled a script that simply had to copy the files I wanted to backup on the dropbox folder. Then I have the files backupped in the cloud and automatically synced on my local machine. Furthermore i had chance to share the backup folder with all my collaborators.

This solution works very well for small projects so I will resume all the steps I followed to install and use dropbox this way. I used an ubuntu machine so I suppose the following steps should work on debian machines.

Prepare the dropbox user

I preferred to have a dedicated user to handle the whole Dropbox daemon and folder so just create it now:

Terminal window

sudo useradd -d /dropbox -m dropbox

It will have the directory /dropbox as home and the name dropbox . You can change these values if you like.

Then you have to set the password for the new user:

Terminal window

sudo passwd dropbox

Choose whatever password you like.

Security concern: If you have ssh access enabled (obviously it is) it's better to disable the ssh access for the new user. So edit the file /etc/ssh/sshd_config and add the rule DenyUsers dropbox , the restart ssh with sudo service ssh restart .

Install the dropbox client

First of all you need to switch to the user created in the previous step, so the Dropbox installer will create the Dropbox folder under its home. At the end you will have /dropbox/Dropbox as the synced folder:

Terminal window

su dropbox

(enter the password for the user dropbox)

Now you're the dropbox user. Be sure to switch to your user folder with cd and let's download and install the daemon.

Terminal window

wget -O dropbox.tar.gz "http://www.dropbox.com/download/?plat=lnx.x86"

for 32bit or

Terminal window

wget -O dropbox.tar.gz "http://www.dropbox.com/download/?plat=lnx.x86_64"

for 64bit.

Extract it:

Terminal window

tar -xvzf dropbox.tar.gz

It will extract to the /.dropbox-dist. folder. Now run the client:

Terminal window

/.dropbox-dist/dropbox

You will see an output like the following:

This client is not linked to any account... Please visit {SOME_URL} to link this machine. [...]

Copy and paste the provided URL in the browser bar of your local machine and it will ask you to enter the password of your dropbox account. This way it is able to authenticate your command line client and start syncing your files. At this point it should have been started its work but our shell is still locked by the client. We need to kill and daemonize it to being able to manage it as a service. Press CTRL+C and get back to your user with the exit command.

Dropbox as a service

At this point we need to define dropbox as a service. So let's create an etc init script . Download my gist

(Edited for length)

Terminal window

Akka workshop at Finn.no

Sjur Millidahl · Finn.no Tech

Akka workshoppers hard at work!

In FINN.no we have experience in running distributed systems, and we often rely on asynchronous communication between our applications. Apache Kafka is the work-horse on the house, and it's doing an excellent job for us.

But this doesn't mean that we shouldn't learn new things - or look at other solutions.

Actors sending messages

A number of Actors communicate to each other using messages. An Actor will process its received messages sequentially, and send messages to other actors asynchronously. It can also keep state, perform side-effects and supervise other actors. This small hand-full of operations makes the Actor Model easy to reason about and within one Actor we need not worry about the complexity of concurrency - one message is processed after another.

The power of the Actor Model comes from combining (often specialized) Actors in a supervised hierarchy. Concurrency and scalability comes naturally as the sending of messages between them are asynchronous. Reliability and uptime comes from a let-it-crash philosophy and supervision done through special Supervisor-Actors.

Arild wrapping heads around the async monad!

The workshop was a hands-on with a few slides between the exercises. You can do the tasks yourself by cloning and checking out the master-branch (for java) or the kotlin-branch.

Åpen fagkveld hjemme hos FINN

Gunn Skinderviken · Finn.no Tech

Tradisjonen tro åpner vi også i år dørene hjemme hos oss i Grensen for å dele våre erfaringer rundt hvordan vi jobber med produktutvikling. Teknologi er i fokus, og vi gleder oss til å vise dere hvordan vi bygger - og holder et av Norges mest besøkte nettsteder vedlike. En kveld er dedikert studenter, og en de mer erfarne.

GDPR

Sosial rekruttering

Universell utforming

Dette ser vi etter når vi ansetter utviklere

Åpen fagkveld 28. februar kl 16-20 (for de som jobber med teknologi)

16:00 Velkommen til FINN ved Nicolai Høge (CTO)

16:30 Mat og drikke

17:00 Sosial rekruttering Hvordan vi tok i bruk våre egne data for å lage FINN Jobbmatch.

17:30 GDPR Samtykke - Hvorfor må vi ha det og hvordan håndterer FINN det i en mikroservices-verden?

17:30 Unleash Verktøy/rammeverk for A/B testing.

18:00 Oppetid Hvordan arbeider vi for høy oppetid i FINN. Ambisjoner, målinger og oppfølgning.

18:00 Podium Frontend i en mikroservices-verden. Hvordan FINN bygger en unison frontend på kryss av microservicer og team med forskjellige eierskap.

Åpen studentkveld 7. mars kl 16-20 (for studenter)

Program (endringer kan komme)

16:00 Velkommen til FINN ved Nicolai Høge (CTO)

16:30 Mat og drikke

17:00 Dette ser vi etter når vi ansetter utviklere

17:30 Sosial rekruttering Hvordan vi tok i bruk våre egne data for å lage FINN Jobbmatch

17:30 Universell utforming - hvorfor og hvordan man skal lage noe som funker for alle. Absolutt ALLE!

18:00 Unleash Verktøy/rammeverk for A/B testing

18:00 Podium Frontend i en mikroservices-verden. Hvordan FINN bygger en unison frontend på kryss av microservicer og team med forskjellige eierskap.

Takk til alle dere som kom.

Karrieresiden vår inneholder mer info om oss.

New PHP library: PHPoAuthUserData

Luciano Mammino · Luciano Mammino (loige)

I recently wrote a new PHP library to simplify the extraction of user data (name , email , id , etc...) from various OAuth providers such as Facebook , Twitter and Linkedin .

Is well know that OAuth 1 and 2 are great standard protocols to authenticate users in our apps. Anyway we often need to go further the authentication process and extract various information about the authenticated users. Unfortunately this is something that is not standardized and obviously each OAuth provider manages user data in very specific manner according to its specific purposes.

So each OAuth provider offer a set of APIs with specific data schemes to allow developers to extract data about the authenticated users.

That's not a big deal if we build apps that adopts a single OAuth provider, but, if we want to adopt more of them, you need to deal with several different APIs and data schemes! Yes, things can get really cumbersome!

Just to make things clearer suppose you want to allow users in your app to sign up with Facebook, Twitter and Linkedin. Probably, to increase conversion rate and speed up the sign up process, you may want to populate the user profile on your app by copying data from the OAuth provider user profile he used to sign up. Yes, you have to deal with 3 different sets of APIs and data schemes! And suppose you would be able to add GitHub and Google one day, that will count for 5 different APIs and data schemes... not so maintainable, isn't it?

The library I wrote is called PHPoAuthUserData . It sits on top of the excellent OAuth library Lusitanian/PHPoAuthLib and aims to resolve the user extraction data problem in the most simple and effective way.

It offers a uniform and (really) simple interface to extract the most interesting and common user data such as Name , Username , Id and so on.

Just to give you a quick idea of what is possible with the library have a look at the following snippet:

/\/ $service is an istance of \OAuth\Common\Service\ServiceInterface (eg. the "Facebook" service) with a valid access token

$extractorFactory = new \OAuth\UserData\ ExtractorFactory ();

The code is available on Github where you will find detailed information on how to install and use the library.

I Hope you will enjoy it and be willing to contribute to the code base. If you want to know more, read the next post that explains how to write an extractor for the library .

Åpen studentkveld hjemme hos FINN

Nicolai Høge · Finn.no Tech

FINN.no skal ha en åpen studentkveld der vi forteller litt om hvordan vi jobber med fokus på teknologi, produkt og brukeropplevelse.

Dette vil skje den 3. november fra klokken 16:00 – 20:00, her i våre lokaler i Grensen 5-7 i Oslo.

Hvem kan melde seg på?

Alle som studerer ved en relevant linje på et universitet eller en høyskole

Våre tre mål for kvelden

Dele kunnskap om hvordan vi jobber

Fortelle litt om hva vi ser etter i kandidater, og hjelpe studentene med å få et innblikk i hvordan reisen er fra student til jobb. Enten det er i FINN eller andre steder.

Vi ønsker selvfølgelig også å profilere FINN Teknologi og Brukeropplevelse som et spennende sted å jobbe, uten at dette er et rekrutteringsarrangement

I disse timene kommer vi til å gi dere et innblikk i hvordan det er å jobbe med Norges mest besøkte nettside.

Vi viser hvordan vi setter brukeren i fokus og jobber med å skape brukeropplevelser som begeistrer.

Hør om hvordan vi jobber i landets ledende interne teknologimiljø. Hvilke teknologier vi benytter, hvordan vi organiserer oss og hva vi ser etter når vi ansetter folk i FINN.

Du vil også få tips om hvordan det er å gå fra studenttilværelsen til å bli en medarbeider i et profesjonelt selskap

Som i alt vi gjør så skal vi også ha det litt gøy, vi bjudar på pizza og øl :-)

Hilsen alle oss i FINN

Disse plassene står og venter på deg:

Takk til alle som kom for en hyggelig kveld. Her er presentasjonene.

Små og hyppige Leveeranser!

Fra Student til FINN

Hva ser vi etter når vi rekrutterer?

So each OAuth provider offer a set of APIs with specific data schemes to allow developers to extract data about the authenticated users.

Luciano Mammino

ANALYSIS

IN BRIEF

Phoebe Sajor, Stack Overflow Blog: Ryan is joined by Jan Liphardt, CEO and co-founder of OpenMind, to chat about the rapidly evolving world of humanoid robotics and what it means for humans, why OpenMind is building an open-source operating system for robots that processes logic in natural language, and how putting Asimov's Laws on the blockchain might be the key to robotics guardrails.

David Loker, Stack Overflow Blog: What specific kind of bugs is AI more likely to generate? Do some categories of bugs show up more often? How severe are they? How is this impacting production environments?

yifanlu, Yifan Lu: I am not a fan of New Year's resolutions, but I do want to do more technical writing this year. So here is a preprint of a paper I wrote on glitching the PS Vita as well as a simple model for reasoning about voltage glitches at a low level .

MapTiler (Martin Mikita), MapTiler Blog: Maps rendered with MapTiler or available as MBTiles file can be uploaded to Amazon S3 and viewed using libraries such as Leaflet, OpenLayers, WebGL Earth, Google Maps API, MapBox.js, etc.

MapTiler (Dominik Zochowski), MapTiler Blog: Fully control styling, behavior, and interaction logic of custom map controls. With MapTiler SDK JS, using native HTML, CSS, and JavaScript, you can fully control the map UI with declarative or programmatic APIs.

PortSwigger Web Security Blog, PortSwigger Web Security Blog: Note: This is a guest post by IT security consultant Adarsh Kumar. I've been using Burp Suite day to day for years, so when Burp AI was introduced, I was curious how it would actually hold up dur

Erin Yepis, Ryan Donovan, Stack Overflow Blog: We're running a survey to understand how people are using AI to learn and whether that's helping, hurting, and replacing tools.

MapTiler (Tomas Pohanka), MapTiler Blog: OpenMapTiles 3.15, the open-source vector tiles map-generation tool, is out now. It enhances the road network, improves water features, and better cartography.

PortSwigger Web Security Blog, PortSwigger Web Security Blog: Note: This is a guest post by pentester and researcher, Tom Stacey (@t0xodile). You'd think that after almost 21 years since its initial public discovery, HTTP Request Smuggling would be barely exploi

Phoebe Sajor, Stack Overflow Blog: Ryan chats with Kevin Peterson, CTO of Bedrock Robotics, about the evolution of self-driving technology and why robotics is now advancing; how real data is still relevant but simulation becomes essential for scale; and the future of robotics in addressing labor shortages and enhancing productivity.

Southern Report

Vol. 1, No. 049 · 2026-03-30

Curated and composed automatically.